Cloud and Mobile Computing

Lab 1 Report

Exploring Cloud Virtualization and Data Center Architecture

Student Name:	Bassel Adel
Group:	SE3
Lab Title:	Cloud Virtualization & Data Center Architecture

Learning Objectives

- Understand the difference between VMs and containers in practice.
- Deploy and compare instances on cloud and local environments.
- Visualize a warehouse-scale architecture and analyze tail latency patterns.
- Experiment with virtualization overhead and container density.

Part A: VMs vs Containers Comparison

In this part, a real Ubuntu VM was launched using Multipass and a Docker container was launched locally. Resource usage was measured in both environments using free -h (memory), ps aux (processes), and df -h (disk) to demonstrate the key differences between full virtualization and containerization.

Multipass VM — Resource Usage

A Multipass Ubuntu 22.04 VM was launched and inspected. The VM runs a full OS stack with many background processes including systemd, sshd, journald, and ModemManager:

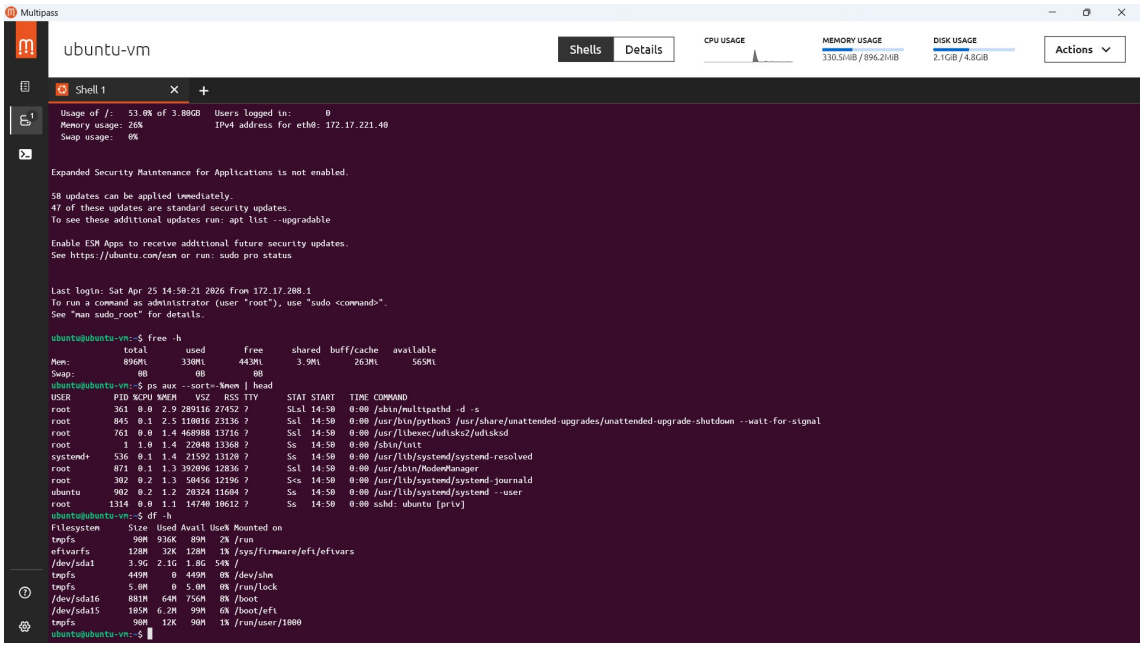

Figure 1: Multipass VM — 896MB dedicated RAM, 330MB used, 10+ background processes, 3.9GB fixed disk

Docker Container — Resource Usage

A Docker container was launched with Ubuntu and the same commands were run. The container shares the host kernel and has minimal overhead:

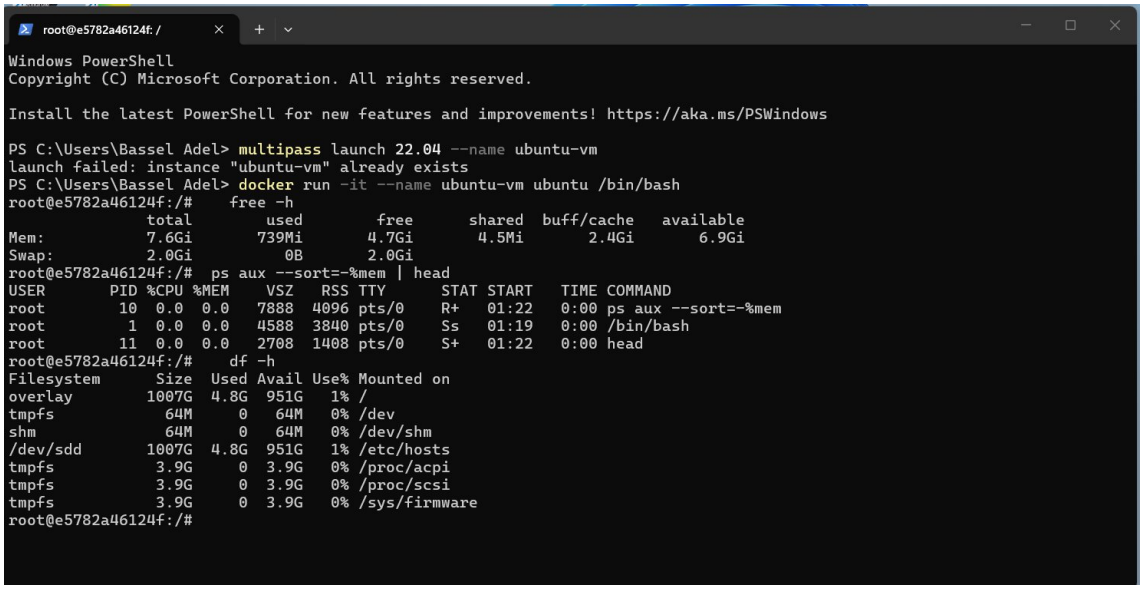

Figure 2: Docker container — only 3 processes (bash, ps, head), shares host's 7.6GB RAM, overlay filesystem

VM vs Container Comparison

Metric	Multipass VM	Docker Container
Total RAM	896MB (dedicated)	7.6GB (shared host)
Memory Used	330MB	739MB
Running Processes	10+ (systemd, sshd, journald...)	Only 3 (bash, ps, head)
Startup Time	~30 seconds	Under 2 seconds
Disk	3.9GB fixed allocated	1007GB shared host (overlay)
OS overhead	Full Ubuntu OS stack	Shares host kernel — minimal
Isolation level	Hardware-level (hypervisor)	Process-level (namespaces)

Part B: Cloud Infrastructure Exploration (AWS EC2)

A t3.micro EC2 instance was launched on AWS Free Tier running Amazon Linux. The Nitro Hypervisor was explored by running dmidecode and htop commands via EC2 Instance Connect in the browser.

EC2 Instance Launch

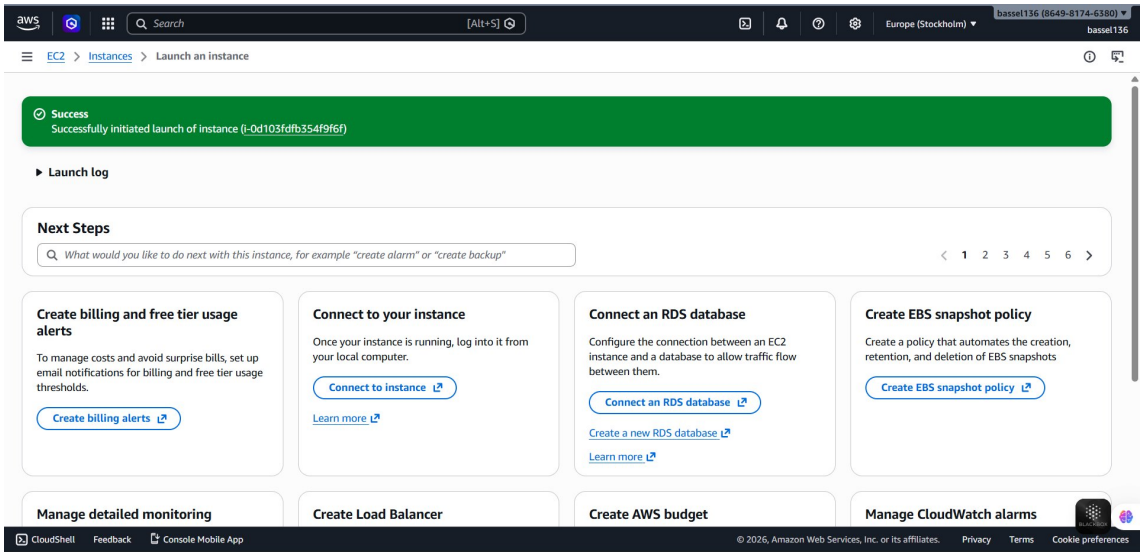

Figure 3: Successful EC2 instance launch on AWS (t3.micro, Europe/Stockholm region)

Nitro Hypervisor & System Info

Running dmesg | grep -i nitro returned an 'Operation not permitted' error, demonstrating that the Nitro hypervisor enforces strict security and blocks direct kernel buffer access. Running sudo dmidecode confirmed AWS infrastructure:

```
Manufacturer: Amazon EC2    |    Product Name: t3.micro    |    Version: Not Specified
```

htop — Process Viewer on EC2

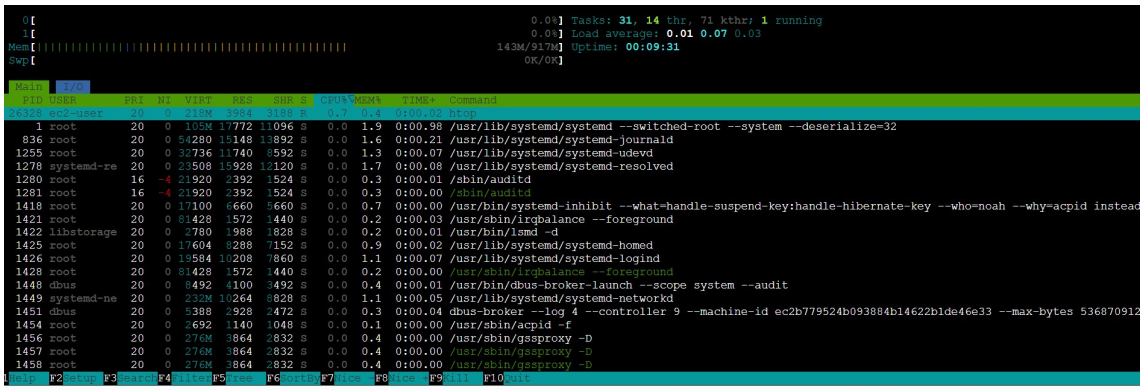

Figure 4: htop on EC2 showing 31 tasks — significantly more than a Docker container's 3 processes

Key Observation

The EC2 VM runs 31 background tasks including systemd, journald, auditd, networkd, and dbus — compared to only 3 processes in the Docker container. This confirms that VMs carry much higher overhead due to running a full OS stack, consistent with the Multipass VM results in Part A.

Part C: Tail Latency Mini-Simulation

A Flask web application was deployed locally with an artificial exponential delay (mean 100ms) to simulate real-world tail latency behavior. 100 sequential HTTP requests were made using a Python benchmarking script.

Flask App Code

```
from flask import Flask
import time, random
app = Flask(__name__)

@app.route('/')
def hello():
    delay = random.expovariate(1/0.1)
    time.sleep(delay)
    return f"Response after {delay:.2f} seconds"

app.run(host='0.0.0.0', port=5000)
```

Benchmark Results

Metric	Value
Total Requests	100
Minimum Latency	2014.08 ms
Maximum Latency	2387.74 ms
Average Latency	2126.29 ms
P90 (Tail Latency)	2217.14 ms
P99 (Tail Latency)	2320.09 ms

Latency Histogram

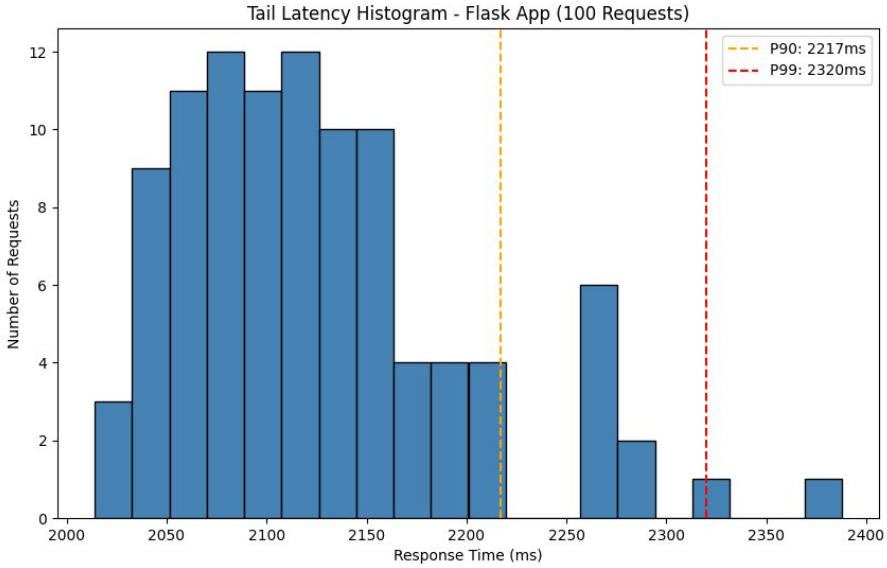

Figure 5: Response time histogram — Orange = P90 (2217ms), Red = P99 (2320ms)

Analysis

The histogram shows most requests completed between 2050–2150ms. However, a small number experienced significantly longer delays reaching up to 2387ms — the classic tail latency pattern. The exponential distribution of delays means that while most responses are fast, a small percentage are disproportionately slow. In warehouse-scale computing, when hundreds of services are called in parallel, the overall response time is determined by the slowest component.

Discussion: VMs vs Containers for Microservices

Based on the real measurements from both Multipass VM and Docker container in this lab, containers are the better choice for microservices architectures:

1. Lightweight and Fast: The Docker container ran with only 3 processes vs 10+ in the VM. Container startup takes under 2 seconds vs ~30 seconds for the VM.

2. Density: Because containers share the host OS kernel, you can run many more containers on the same hardware. The VM dedicated 896MB RAM for its OS alone.

3. When VMs are preferred: VMs provide stronger isolation (hardware-level via hypervisor) and are better for running untrusted code, different OS types, or legacy applications.

Discussion Questions

Q: Why does AWS split Nitro into hardware components?

> **A:** AWS splits Nitro into dedicated hardware components (Nitro Cards for networking/storage, Nitro Security Chip, and Nitro Hypervisor) to offload virtualization tasks from the main CPU. This allows EC2 instances to use nearly all CPU and memory for customer workloads rather than hypervisor overhead, improving performance while enhancing security.

Q: In which scenarios would you use VMs over containers?

> **A:** VMs are preferred when: (1) strong security isolation is required between tenants, (2) different operating systems are needed per workload, (3) running legacy applications that depend on full OS features, or (4) compliance requirements mandate hardware-level isolation.

Q: How does tail latency change with the number of parallel calls?

> **A:** Tail latency worsens significantly as parallel calls increase. If one request has a 1% chance of hitting a slow response, then with 100 parallel calls, the probability that at least one is slow approaches nearly 100%. The overall response time becomes dominated by the slowest component — exactly what the histogram demonstrates.